VERIQTA

KUBERNETES PRODUCTION OUTAGES

PRODUCTION IMPACT REAL USERS AT RISK!

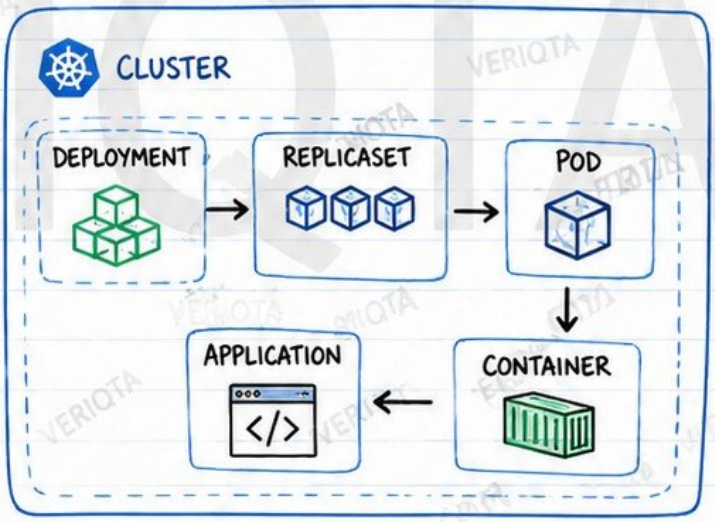
EVERY ENGINEER SHOULD KNOW

VOLUME 1: WORKLOAD AND POD FAILURES

- PRODUCTION READY
- SENIOR ENGINEER FOCUS
- REAL OUTAGES
- PRACTICAL WORKFLOWS

KEY THEMES

- POD LIFECYCLE FAILURES
- CONTAINER STARTUP PROBLEMS
- RESOURCE EXHAUSTION
- DEPLOYMENT INCIDENTS
- APPLICATION HEALTH FAILURES
- PRODUCTION INVESTIGATION WORKFLOWS



WHAT THIS VOLUME COVERS

- WHY WORKLOADS FAIL IN PRODUCTION
- HOW TO INVESTIGATE LIKE A SENIOR ENGINEER
- REAL OUTAGE SCENARIOS & ROOT CAUSES
- PROVEN DEBUGGING WORKFLOWS
- PREVENTION, DETECTION & RECOVERY
- BEST PRACTICES & DESIGN DECISIONS

PRODUCTION NOTE

KUBERNETES CAN KEEP THE INFRASTRUCTURE HEALTHY... WHILE YOUR USERS STILL EXPERIENCE A COMPLETE OUTAGE.

ALWAYS CORRELATE:

K8s STATE + APP HEALTH + DEPENDENCIES + USER IMPACT

HOW KUBERNETES WORKLOAD OUTAGES DEVELOP

1. UNDERSTAND THE LEVELS

FAILED CONTAINER

The process inside the container crashes or exits with an error. Kubernetes restarts the container inside the same Pod (restartPolicy).

FAILED POD

One or more containers in the Pod keep failing and the Pod cannot run workloads as expected. Kubernetes may recreate the Pod in a new attempt.

FAILED APPLICATION (SERVICE IMPACT)

Users experience errors, timeouts or complete unavailability even if some Pods are running. The Service or business functionality is broken.

2. HOW CONTROLLERS RESPOND

DEPLOYMENT

Detects ReplicaSet issues and creates new ReplicaSets or rolls back.

REPLICASET

Ensures the desired number of Pods match the desired state.

KUBELET (ON NODE)

Monitors containers and restarts them based on restart policy.

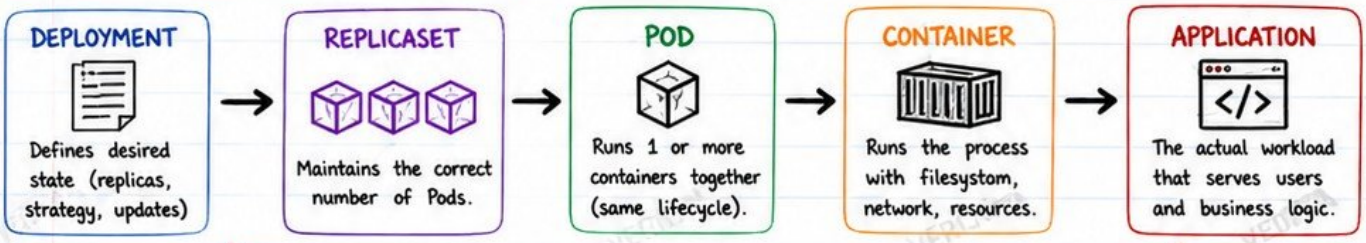
NODE CONTROLLER

Detects node failures and evicts or reschedules the Pods on healthy nodes.

3. WHY A RUNNING POD ≠ A HEALTHY SERVICE

- ☑ Application inside the container may be slow or stuck.
- ☑ Readiness probe may be passing incorrectly.
- ☑ External dependencies (DB, Cache, Queue, API) may be down.
- ☑ The Pod may not be receiving traffic (Service misconfig).
- ☑ Resource starvation can cause high latency or failures.
- ☑ Business logic errors or data issues are not visible to K8s.

4. THE WORKLOAD FAILURE PATH



PRODUCTION NOTE

KUBERNETES CAN KEEP THE INFRASTRUCTURE HEALTHY (NODES UP, PODS RUNNING, CONTROLLERS HAPPY) WHILE YOUR USERS STILL EXPERIENCE A COMPLETE OUTAGE.

ALWAYS CORRELATE: K8s STATE + APP HEALTH + DEPENDENCIES + USER IMPACT

THE SENIOR ENGINEER'S POD INVESTIGATION WORKFLOW

GOAL: Quickly identify root cause and restore service with minimal risk.

1. **CONFIRM USER IMPACT**
   Verify alerts, dashboards and real user experience.
2. **CHECK DEPLOYMENT STATUS**
   Ensure the Deployment/ReplicaSet is progressing as expected.
3. **INSPECT POD STATES**
   Check Pod phase, restarts and container statuses.
4. **REVIEW EVENTS**
   Events reveal what Kubernetes is trying to tell you.
5. **COMPARE DESIRED vs AVAILABLE**
   Spot gaps between desired replicas and actual available replicas.
6. **INSPECT LOGS**
   Check current and previous logs for clues and error patterns.
7. **CHECK PROBES & RESOURCES**
   Validate liveness/readiness probes, CPU, memory and limits.
8. **REVIEW RECENT CHANGES**
   Identify what changed before the outage (deploys, configs, secrets).
9. **VALIDATE APPLICATION DEPENDENCIES**
   Check DB, cache, queues, APIs, DNS, storage and network.
10. **RECOVER SAFELY**
    Apply fix, monitor closely and confirm service is stable before closing.

REMEMBER:
A running Pod does not always mean a healthy application.

INVESTIGATION PRINCIPLES

- ☑ Start broad, then go deep.
- ☑ Collect evidence before making changes.
- ☑ Follow the sequence, don't jump steps.
- ☑ Restore service first, then do RCA.

CORE COMMANDS

```
kubectl get deploy,rs,pods -n production
# View deployments, ReplicaSets and Pods
# in the production namespace

kubectl describe pod <pod-name> -n production
# Detailed pod information: events,
# status, probes, resources

kubectl get events -n production \
  --sort-by=.lastTimestamp
# View events sorted by time
#  (oldest to newest)

kubectl logs <pod-name> -n production --previous
# Fetch logs from the previous container
# instance (before the latest restart)
```

SENIOR TIP

80% of outages are caused by changes, misconfigurations or dependencies.
The faster you follow this workflow, the faster you restore trust.

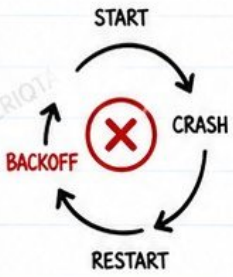
CRASHLOOPBACKOFF

POD STATUS
CrashLoopBackOff

1. WHAT DOES IT MEAN?

CrashLoopBackOff means the container inside the Pod is crashing after start and Kubernetes is repeatedly trying to restart it.

After several failures, Kubernetes slows down the restart attempts using exponential backoff.



2. CONTAINER RESTART BACKOFF BEHAVIOUR

ATTEMPT	RESTART DELAY (APPROX)	NOTES
1	10 seconds	Quick restart
2	20 seconds	Backoff increases
3	40 seconds	More delay
4	80 seconds	Exponential backoff
5+	5 minutes (max)	Stays at max delay

Kubernetes keeps retries, but slows down to avoid overloading the node and the system.

3. COMMON CAUSES

APPLICATION STARTUP FAILURES
- App exits on startup
- Uncaught exceptions
- Fatal misconfiguration
- Port already in use

INVALID COMMANDS & ENTRYPOINTS
- Wrong command
- Missing arguments
- Bad entrypoint script
- Exit immediately

MISSING CONFIGURATION
- Missing ConfigMap
- Missing Secret
- Environment variables
- Required files not found

FAILED DEPENDENCY CONNECTIONS
- Database unavailable
- DNS resolution failure
- Service not reachable
- Network issues

PERMISSION ERRORS
- Cannot write to volume
- Read-only filesystem
- User lacks permissions
- Root vs non-root issues

INCORRECT HEALTH PROBES
- Liveness probe too aggressive
- Readiness probe misconfigured
- Probes hitting wrong endpoint
- Startup time too slow

RESOURCE LIMITS (INDIRECT CAUSE)
- OOMKilled due to low memory
- CPU throttling on startup
- Insufficient resources

REAL-WORLD SCENARIO

A simple config typo in an environment variable causes the app to crash immediately. Kubernetes keeps restarting the container, but the real problem is the bad config, not Kubernetes.

4. HOW TO INVESTIGATE

1. `kubectl get pods -n production` # Check status
2. `kubectl describe pod <pod-name> -n production` # Events, last state, reason
3. `kubectl logs <pod-name> -n production --previous` # Logs from the previous crash
4. `kubectl get events -n production --sort-by=.lastTimestamp` # Recent events
5. Check configuration, env, secrets and dependencies
6. Fix the root cause and restart the workload

DEBUGGING INSIGHT

CrashLoopBackOff is NOT the root cause.
It is Kubernetes slowing repeated restart attempts.

CORE COMMANDS

```
kubectl get deploy,rs,pods -n production                  # View deployments, ReplicaSets and Pods
kubectl describe pod <pod-name> -n production             # Get detailed Pod information and events
kubectl get events -n production --sort-by=.lastTimestamp # See recent events (oldest to newest)
kubectl logs <pod-name> -n production --previous          # View logs from the previous crash
```

CRASHLOOPBACKOFF INVESTIGATION AND RECOVERY

1. INVESTIGATION WORKFLOW

POD STATUS
Is the Pod crashing?

→

PREVIOUS LOGS
What happened before it crashed?

→

EXIT CODE
How did the container exit?

→

EVENTS
What is Kubernetes telling us?

→

CONFIGURATION
Is the config, command, env, or secret wrong?

→

DEPENDENCIES
Are external services available?

2. IMPORTANT CHECKS (CORE COMMANDS)

```
$ kubectl logs <pod> --previous
```
Logs from the previous container instance (before the latest crash)

```
$ kubectl describe pod <pod>
```
Detailed info: events, status, conditions, probes, mounts, env, etc.

```
$ kubectl get pod <pod> -o jsonpath='
{.status.containerStatuses[*].lastState}'
```
See last state: exit code, reason, and finished time

3. WHAT TO LOOK FOR IN "kubectl describe pod"

- State: Waiting (CrashLoopBackOff)
- Last State: Terminated
  - Reason: Error / OOMKilled / Completed
  - Exit Code: (not 0)
  - Started / Finished timestamps
- Events: Look for warnings and errors
- Probes: Liveness / Readiness failures
- Mounts & Volumes: Permission / not found
- Environment: Missing values
- Image: Pull issues
- Node: Resource pressure

4. COMMON EXIT CODES (QUICK GUIDE)

EXIT CODE	MEANING	COMMON CAUSES
0	Success	Should not crash
1	Generic Error	App error, bad config
2	Misuse of shell	Bad command/script
126	Permission denied	Entry point not executable
127	Command not found	Wrong command/entrypoint
137	OOMKilled	Memory limit exceeded
143	SIGTERM	Graceful termination
255	Unknown	Unexpected runtime error

5. RECOVERY DECISIONS

FIX THE APPLICATION CONFIGURATION	Correct env vars, config files, paths, or arguments.
RESTORE THE MISSING SECRET	Recreate or update the Secret / ConfigMap.
CORRECT THE STARTUP COMMAND	Fix command, args, or entrypoint.
TEMPORARILY DISABLE A DESTRUCTIVE PROBE	Prevent restart storms while you diagnose.
ROLL BACK THE RELEASE	Revert to last known good version.

AVOID REPEATEDLY DELETING THE POD WITHOUT IDENTIFYING THE CAUSE

Deleting the Pod only hides the evidence and delays the fix.
Always capture logs and events before taking action.

PRO TIP

Always check previous logs FIRST. The latest container may crash too fast to show the real error.

FOCUS ON ROOT CAUSE

CrashLoopBackOff is a SYMPTOM. Your goal is to find why the application fails on startup.

SYMPTOM ≠ PROBLEM

SENIOR HABIT

Collect → Analyze → Fix → Verify
Document what you learn so the team doesn't repeat the same mistake.

IMAGEPULLBACKOFF AND ERRIMAGEPULL

POD STATUS
ImagePullBackOff
ErrImagePull

These errors mean Kubernetes cannot download the container image from the registry.

1. COMMON CAUSES

INCORRECT IMAGE REPOSITORY
Wrong registry URL or repository path.

MISSING OR INVALID IMAGE TAG
Tag does not exist or was typo'd.

REGISTRY AUTHENTICATION FAILURE
Invalid username, password or token.

EXPIRED IMAGE PULL CREDENTIALS
Docker config secret or token expired.

REGISTRY RATE LIMITING
Too many pulls from Docker Hub or private registry.

PRIVATE REGISTRY NETWORK FAILURE
DNS issue, firewall, proxy or outbound connectivity problem.

MISSING IMAGEPULLSECRETS
Secret not created, not referenced, or wrong namespace.

ARCHITECTURE MISMATCH
Image built for a different CPU architecture (e.g., ARM vs AMD64).

KEY SYMPTOMS

- ☑ Pod stuck in ImagePullBackOff / ErrImagePull
- ☑ Events show pull failures
- ☑ Containers never start
- ☑ No logs from the container
- ☑ Backoff increasing over time

2. PRODUCTION WORKFLOW

POD EVENT	IMAGE REFERENCE	REGISTRY ACCESS	SECRET	NODE CONNECTIVITY
Check event message for exact error.	Verify image repository and tag in the spec.	Can the node reach the registry and is it available?	Is the correct imagePullSecret present and valid?	Can the node download the image without restriction?

3. QUICK CHECK COMMANDS

```
kubectl describe pod <pod-name> -n <ns>          # Check events and reason
kubectl get pod <pod-name> -n <ns> -o wide       # See node and status
kubectl get secret -n <ns>                       # Verify imagePullSecrets exist
kubectl get secret <secret-name> -n <ns> -o yaml # Validate secret config
curl —I https://<registry>/v2/                   # Test registry access (from node)
```

PRODUCTION TIP

Always pin image tags to immutable versions (e.g., 1.2.3) instead of latest to avoid unexpected pull failures.

IMPORTANT

ImagePullBackOff is a SYMPTOM.
Your goal is to find why the image cannot be pulled, not just restart the Pod.

CONTAINERS STUCK IN CREATING

POD STATUS
ContainerCreating

The Pod has been scheduled to a node, but the container is not ready to run yet.

1. WHY PODS REMAIN IN CONTAINERCREATING

IMAGE DOWNLOAD DELAY
Large images, slow registry, or poor network causes long pulls.

CNI NETWORK SETUP FAILURE
Plugin errors, IP allocation issues, or node network not ready.

VOLUME MOUNT FAILURE
PVC not bound, missing volume, or driver/plugin issues.

SECRET OR CONFIGMAP REFERENCE FAILURE
Missing secret/config, wrong key, or permission denied.

CONTAINER RUNTIME PROBLEMS
containerd/CRI-O/Docker issues, sockets down or misconfigured.

SANDBOX CREATION ERRORS
Pause image pull failure, invalid sandbox config, or runtime crash.

NODE DISK PRESSURE
Not enough disk space for image, logs, or container layers.

2. WHAT'S HAPPENING UNDER THE HOOD?

POD SCHEDULED → **POD SENT TO NODE** → **NODE PREPARATION** → **CONTAINER START**

- **POD SCHEDULED**: Scheduler picks the node.
- **POD SENT TO NODE**: Kubelet receives the Pod spec.
- **NODE PREPARATION**: Pull image, setup network, mount volumes, verify secrets/config.
- **CONTAINER START**: Create sandbox and start the container.

QUICK INVESTIGATION CHECKLIST

- [x] kubectl describe pod <pod> -n <ns>
- [x] Check Events for exact reason
- [x] Verify PVC is Bound
- [x] Check secrets/config existence
- [x] Check node disk and runtime status
- [x] Look for image pull or CNI errors

USEFUL COMMANDS

```
kubectl describe pod <pod>
kubectl get events -n <ns>
--sort-by=.lastTimestamp
kubectl get pods -o wide
kubectl logs <pod> -n <ns>
(if available)
kubectl get pvc
kubectl get nodes
```

3. KEY DISTINCTION

PENDING
- Pod not scheduled yet
- Scheduler couldn't find a suitable node
- Causes: resources, affinity, taints, PVC, quotas, etc.

VS

CONTAINERCREATING
- Pod is already scheduled to a node
- Node is preparing the Pod to run
- Issues are on the node (runtime, network, storage, secrets, etc.)

SENIOR ENGINEER TIP
ContainerCreating means the problem is NOT with scheduling. Focus on the node, runtime, network, storage and dependencies.

PRODUCTION NOTE
Most ContainerCreating issues are fixable without deleting the Pod. Read the Events before acting.

PODS STUCK IN PENDING

A Pending Pod means Kubernetes could not schedule the Pod onto any node.

1. SCHEDULER-RELATED CAUSES

INSUFFICIENT CPU OR MEMORY
No node has enough available CPU or memory to fit the Pod.

NODE SELECTOR MISMATCH
No node matches the nodeSelector labels required by the Pod.

REQUIRED AFFINITY RULES
NodeAffinity / PodAffinity rules cannot be satisfied on any node.

UNTOLERATED TAINTS
All matching nodes have taints that the Pod does not tolerate.

UNBOUND PERSISTENTVOLUMECLAIM
PVC is not bound, so the Pod cannot be scheduled.

POD TOPOLOGY CONSTRAINTS
TopologySpreadConstraints or zone/region rules cannot be met.

NAMESPACE QUOTA EXHAUSTION
Namespace has hit CPU, memory, storage or object quotas.

MAXIMUM POD COUNT REACHED
Node has hit the max number of Pods allowed.

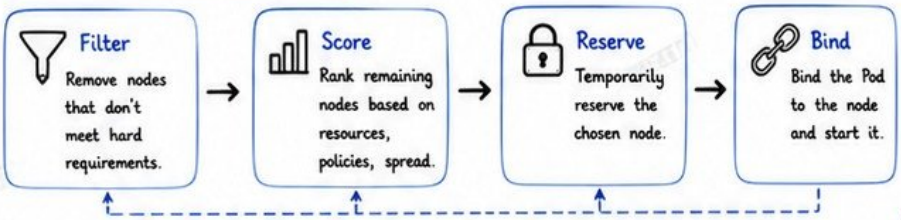
2. PRODUCTION WORKFLOW

SCHEDULER EVENT	CAPACITY (NODES)	PLACEMENT RULES	STORAGE (PVC)	QUOTAS (NS)
Check events for exact reason.	Verify node resources availability.	Check selectors, affinity, taints and topology.	Ensure PVCs are bound and ready.	Validate namespace quotas.

3. QUICK INVESTIGATION COMMANDS

```
kubectl get pods -n <ns>                                  # See Pending Pods
kubectl describe pod <pod> -n <ns>                        # Check scheduler events
kubectl get events -n <ns> --sort-by=.lastTimestamp       # Recent events
kubectl get nodes -o wide                                 # Node capacity overview
kubectl describe nodes                                    # Detailed node info
kubectl get pvc -n <ns>                                   # PVC status
kubectl get resourcequota -n <ns>                         # Quota usage
kubectl get limitrange -n <ns>                            # Limits & defaults
kubectl get pods -A --field-selector=status.phase=Pending # Cluster-wide pending
```

4. HOW SCHEDULER DECIDES



SENIOR TIP

- ☑ Always read the Events first.
- ☑ Don't guess -- the scheduler tells you exactly why.
- ☑ Fix the root cause, then let the controller reschedule.
- ☑ Avoid manual workarounds unless it's an emergency.

KEY DISTINCTION

Pending:
Scheduling not successful.
(Pod not assigned to any node)

VS

ContainerCreating:
Scheduling successful,
but node-side setup is in progress.
(Pod already assigned to a node)

EXAMPLE EVENT MESSAGE

```
0/3 nodes are available:
1 Insufficient cpu.
1 node(s) had untolerated taint{dedicated=apps:NoSchedule}.
1 node(s) didn't match node selector.
```

OOMKILLED CONTAINERS

POD STATUS
OOMKilled
Exit Code: 137

OOMKilled means the Linux kernel killed the container because it used more memory than allowed.

1. HOW LINUX MEMORY ENFORCEMENT WORKS

- Application allocates memory
- ↓ Container runs inside a cgroup (memory limit enforced here)
- ↓ If usage > memory limit
- ↓ Kernel OOM killer terminates the container
- ↓ Kubernetes reports: OOMKilled
  Exit Code: 137

2. EXIT CODE 137

Exit Code 137 = 128 + 9
Signal 9 (SIGKILL) sent by the kernel

Not an application error. It's the kernel protecting the node.

3. MEMORY REQUESTS vs LIMITS

REQUEST (memory)	LIMIT (memory)
• Used by scheduler to place the Pod • Not a hard limit • Too low = Pod may be packed on the node	• Hard limit enforced by cgroup • Exceeding it triggers OOMKill • Too high = Risk of node memory pressure

Requests influence placement.
Limits enforce memory boundaries.

4. COMMON CAUSES OF OOMKILLED

APPLICATION HEAP GROWTH
Java, .NET, Python or Node apps may grow heap beyond the container limit.

MEMORY LEAKS
Unreleased objects, caches, or connections slowly consume memory.

SIDE CAR CONSUMPTION
Envoy, Fluent Bit, Istio, or other sidecars may consume more memory than expected.

NODE-LEVEL MEMORY PRESSURE
Other Pods on the node cause reclaim pressure or eviction.

5. INVESTIGATION WORKFLOW

POD STATUS → PREVIOUS LOGS → USAGE TRENDS → NODE PRESSURE → ROOT CAUSE

USEFUL COMMANDS

```
kubectl describe pod <pod> -n <ns>                  # Check state, reason, events
kubectl logs <pod> --previous -n <ns>               # Logs from last terminated container
kubectl top pod <pod> -n <ns>                       # Live memory usage
kubectl top node                                    # Node memory pressure
kubectl describe node <node>                        # Check for memory pressure/evictions
kubectl get events -n <ns> --sort-by=.lastTimestamp # Recent OOM events
```

6. WHY INCREASING THE LIMIT IS NOT ALWAYS THE FIX

INCREASING MAY HELP WHEN:
- Legitimate traffic growth
- Higher baseline memory need
- Application requires more heap
- Node has enough free memory

INCREASING CAN HURT WHEN:
- There is a memory leak
- App is inefficient
- Analyer is the real consumer
- It causes other Pods to get OOMKilled
- Node memory becomes overcommitted

SENIOR ENGINEER TIP

Confirm whether memory usage is expected growth, traffic-driven pressure, a leak, or poor application sizing before changing limits.

7. EXAMPLE: OOMKILLED POD EVENT

```
Events:
Type      Reason      Age    From       Message
--------  ---------   -----  ---------  --------------------
Warning   OOMKilled   2m     kubelet    Killing container <app>
                      Reason:  OOMKilled
                      Exit Code:  137
                      Container exceeded memory limit.
```

KEY CHECKLIST

- ☑ Check Pod status and events
- ☑ Review previous logs
- ☑ Analyze memory usage trends
- ☑ Identify who consumes memory
- ☑ Check node memory pressure
- ☑ Fix root cause, then adjust limits

CPU THROTTLING WITHOUT POD FAILURE

Kubernetes shows the Pod as Running, but Linux is throttling the CPU.

1. SYMPTOMS YOU WILL SEE

- Increased latency
- Slow health checks
- Request timeouts
- Reduced throughput
- No pod restart
- Normal-looking CPU usage graphs

2. WHAT IS CPU THROTTLING?

When a container uses more CPU than its limit, Linux CFS (Completely Fair Scheduler) throttles it.

Container wants more CPU → CPU limit reached → CPU time is paused periodically

KEY POINT

Throttling slows your app down without killing the container.

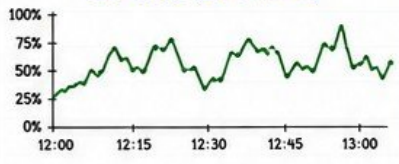
3. COMMON CAUSES

RESTRICTIVE CPU LIMITS	BURSTY APPLICATIONS	GARBAGE COLLECTION	HIGH CONCURRENCY	POORLY CONFIGURED AUTOSCALING	SIDECAR CPU CONTENTION
Limit too low for the workload.	Short spikes exceed the limit often.	GC pauses cause CPU spikes.	Too many requests at the same time.	HPA doesn't scale fast enough.	Sidecar uses CPU needed by app.

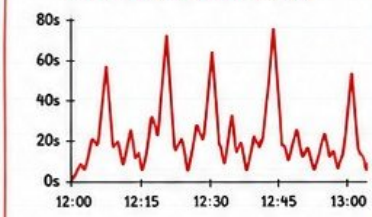
4. CPU THROTTLING OBSERVED

METRICS TO CHECK

- ☑ container_cpu_cfs_throttled_seconds_total
- ☑ container_cpu_cfs_periods_total
- ☑ container_cpu_usage_seconds_total
- ☑ kube_pod_container_resource_limits_cpu_cores



Looks fine... - ...but performance is poor!



High throttling = slow app

5. WHAT TO CHECK

- ☑ Compare CPU usage vs CPU limit.
- ☑ Check throttling metrics.
- ☑ Review application logs for slow responses.
- ☑ Identify code paths causing CPU spikes.
- ☑ Validate autoscaling thresholds.
- ☑ Check sidecar resource usage.
- ☑ Load test to reproduce under pressure.

6. HOW TO FIX

- → Increase CPU limit (carefully).
- → Improve application efficiency.
- → Optimize garbage collection settings.
- → Enable or tune autoscaling.
- → Right-size or separate sidecars.
- → Use CPU requests that reflect real usage.

High throttled seconds with normal CPU usage = throttling issue

PRODUCTION LESSON

A pod can remain Running while Linux throttling makes the application practically unavailable.

WHY THIS IS DANGEROUS

- No restarts = no alerts from pod failures
- Users suffer, but monitoring looks "normal"
- SLI/SLO violations without obvious root cause

SENIOR ENGINEER TIP

Always check throttling metrics when performance is bad but pods look healthy.

7. USEFUL COMMANDS

```
kubectl top pod <pod> -n <ns>                                          # Check current CPU usage
kubectl describe pod <pod> -n <ns>                                     # Events may mention throttling
kubectl get --raw /metrics | grep container_cpu_cfs_throttled_seconds_total   # Check throttling metrics
kubectl get pod <pod> -n <ns> -o jsonpath='{.spec.containers[*].resources}'   # Review CPU requests and limits
```

REMEMBER

Throttling is silent.
Measure it, don't guess it.

LIVENESS PROBE FAILURE CAUSES A RESTART STORM

A liveness probe that is too aggressive or poorly designed can kill healthy pods and create a restart storm.

1. COMMON CAUSES

INCORRECT PROBE PATH
The endpoint does not exist or returns an error (404/5xx).

PROBE TIMEOUT TOO SHORT
Application needs more time to respond than the probe allows.

SLOW APPLICATION STARTUP
App is not ready yet, but liveness probe starts killing it.

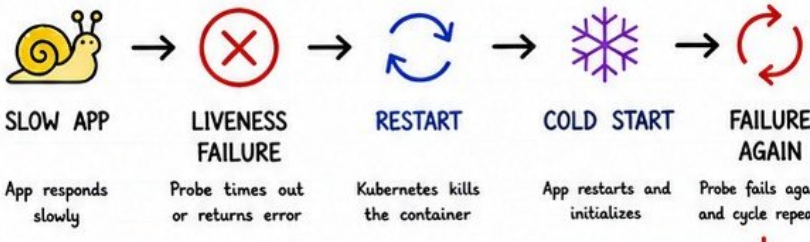
DEPENDENCY CHECKS INSIDE LIVENESS PROBES
Checking database, Redis, or external services can cause false failures.

CPU THROTTLING
High throttling delays responses, causing probe timeouts.

TEMPORARY DATABASE LATENCY
Short spikes in DB latency can make the probe fail.

TOO-SMALL FAILURE THRESHOLDS
One or two failures trigger a restart even during normal variance.

2. FAILURE CYCLE (RESTART STORM)



RESULT: High error rate, downtime, and user impact.

3. WHAT TO CHECK

- ☑ kubectl describe pod <pod> → check Events for probe failures
- ☑ kubectl logs <pod> --previous → see why it crashed
- ☑ Check livenessProbe configuration in Deployment/StatefulSet
- ☑ Test the probe endpoint from inside the container
- ☑ Review application startup time and dependencies
- ☑ Check CPU throttling and resource limits
- ☑ Look at dashboard/alerts around the failure time

USEFUL COMMANDS

```
kubectl describe pod <pod>
kubectl get events -n <ns>
--sort-by=.lastTimestamp
kubectl logs <pod> --previous
kubectl exec -it <pod> --
curl -v http://localhost:8080/health
kubectl top pod <pod>
```

4. BETTER PRACTICES

Use liveness probe only to detect deadlocks or unrecoverable states.	Give enough initialDelaySeconds for slow startup apps.	Keep probes lightweight. Avoid calling external dependencies.	Tune timeoutSeconds, periodSeconds, and failureThreshold carefully.	Monitor and alert on probe failures before users complain.

SENIOR ENGINEER TIP

Liveness probe should answer:
"Is the process alive and responsive?"
It should NOT answer:
"Are all dependencies healthy?"
Use readiness probe for dependency checks.

PRODUCTION FAILURE

A badly designed liveness probe can turn a temporary slowdown into a full outage.

IMPACT

- Pods keep restarting
- Traffic drops
- SLA breaches
- Customer frustration

REMEMBER

Design probes like a scalpel, not a hammer.
Be precise. Be patient.
Protect stability.

READINESS PROBE REMOVES EVERY POD FROM SERVICE

A readiness probe that fails causes Kubernetes to stop sending traffic to the Pod — even if the container is Running.

1. LIVENESS vs READINESS

LIVENESS PROBE	READINESS PROBE
Is the container alive?	Is the container ready to serve traffic?
If it fails → Kubernetes restarts the container.	If it fails → Kubernetes removes the Pod from Service endpoints.

Liveness = Should I restart you?
Readiness = Should I send traffic to you?

2. WHY HEALTHY CONTAINERS MAY RECEIVE NO TRAFFIC

Pod Running → Readiness Probe Fails → Removed from Service Endpoints → No Traffic to Pod

The container is Running, but the application is invisible to the Service.

3. COMMON CAUSES

INCORRECT READINESS ENDPOINT
Wrong path, port, or status code causes the probe to fail.

EXTERNAL DEPENDENCY FAILURES
Database, Redis, APIs, or queues down → readiness fails.

PROBE TIMEOUT OR THRESHOLD ERRORS
Timeout too short or failureThreshold too low → false failures.

SERVICE ENDPOINT REMOVAL
Kubernetes removes the Pod IP from the Endpoints list.

ROLLING DEPLOYMENT DEADLOCKS
New Pods never become ready, so old Pods are terminated → zero capacity.

4. INVESTIGATION COMMANDS

Check Service Endpoints

```
kubectl get endpoints <service>
```

If the endpoints list is empty, no Pods are ready.

Check Pod Events & Probes

```
kubectl describe pod <pod>
```

Look for readiness probe failures, timeouts, and reason messages.

Check Pod Conditions

```
kubectl get pod <pod> \
-o jsonpath='{.status.conditions}'
```

Look for Ready status:
"status": "False", "reason": "..."

5. HOW TRAFFIC FLOW IS AFFECTED

Users → Service (Load Balancer) → Service Endpoints (Only Ready Pods) → Ready Pods Receive Traffic

If no Pods are Ready → Endpoints list is empty → Service has nowhere to send traffic.

6. BEST PRACTICES

- ☑ Use a lightweight readiness endpoint.
- ☑ Do not depend on external systems for basic readiness.
- ☑ Set reasonable timeoutSeconds and failureThreshold.
- ☑ Use startupProbe for slow-starting applications.
- ☑ Monitor readiness failures with alerts.
- ☑ Test probes during load and dependency failures.

SENIOR INSIGHT

A readiness probe should reflect customer readiness, not every downstream dependency. Keep it simple and reliable.

7. QUICK CHECKLIST

- ☑ Pod is Running but not Ready?
- ☑ Readiness probe endpoint correct?
- ☑ External dependencies healthy?
- ☑ Timeouts and thresholds reasonable?
- ☑ Service endpoints list has addresses?

PRODUCTION FAILURE

A bad readiness probe can make an entire application disappear without crashing a single container.

KEY TAKEAWAY

Readiness controls traffic. If every Pod is Not Ready, your Service has zero capacity.

STARTUP PROBE MISCONFIGURATION

Startup probes give slow-starting applications time to initialize before liveness and readiness probes start judging them.

1. WHY STARTUP PROBES EXIST

- Prevent premature restarts during application initialization.
- Allow enough time for the app to become ready to respond.
- Separate initialization phase from ongoing health monitoring.
- Protect slow or complex startups without disabling liveness probes.

2. EXAMPLES OF SLOW STARTUP SCENARIOS

JVM & LARGE FRAMEWORKS Class loading, JIT compilation, Spring Boot / Quarkus / .NET startup.	Can take 30s – 5+ min
DATABASE MIGRATIONS Schema migrations, index creation, or data transformations.	Can take Minutes
CACHE WARMUP & EXTERNAL CALLS Loading configurations, calling external APIs, warming caches.	Can take Minutes
MESSAGE QUEUE / CONNECTORS Connecting to Kafka, RabbitMQ, or other brokers.	Can take Minutes

3. PROBE INTERACTION FLOW

CONTAINER STARTS
↓
STARTUP PROBE
Runs until it succeeds
↓
LIVENESS PROBE
Ensures the container stays alive
↓
READINESS PROBE
Determines if Pod receives traffic

Liveness and readiness probes do NOT run until the startup probe has succeeded.

4. STARTUP PROBE MISCONFIGURATIONS

MISCONFIGURATION	IMPACT	EXAMPLE
No startup probe for slow app	Liveness probe hits too early → container killed → restart storm.	`livenessProbe:` `  initialDelaySeconds: 10  # too short`
Startup failureThreshold too low	Initialization takes longer than allowed → container restarts unnecessarily.	`startupProbe:` `  failureThreshold: 10  # too low`

Startup period too short	App still initializing → probe fails → Kubernetes gives up.	`startupProbe:`
		`  periodSeconds: 2`
		`  failureThreshold: 10  # 20s total`

Excessively generous window	App can be unhealthy for a long time before Kubernetes detects failure.	`startupProbe:`
		`  periodSeconds: 30`
		`  failureThreshold: 60  # 30 min window!`

Startup probe hides permanent failures	App never becomes healthy but gets too much time → masks real issues.	`startupProbe:` `  failureThreshold: 1000  # hides problems`

5. GOOD STARTUP PROBE PRACTICES

- Match the probe timing to realistic startup duration.
- Use a tight, but fair window.
- Check an endpoint that proves the app is truly initialized.
- Avoid extremely long timeouts that hide failures.
- Monitor startup duration in production.
- Log clear readiness signals from the application.

SENIOR TIP

Start with real startup timings from logs and metrics. Tune the startup probe, not the liveness probe, to handle slow initialization.

6. EXAMPLE: RECOMMENDED STARTUP PROBE CONFIG

```
startupProbe:
  httpGet:                              # Check a lightweight startup endpoint
    path: /actuator/health/startup
    port: 8080
  periodSeconds:    5                   # 5s interval
  failureThreshold: 30                  # 30 * 5 = 150s total window
  timeoutSeconds:   2                   # 2s timeout per check
```

CHECKLIST

- Does this app really need a startup probe?
- Are startup timings based on real data?
- Is the endpoint truly indicating startup complete?
- Is the window tight but realistic?
- Will this mask permanent failures?

7. OBSERVE & VALIDATE

- Monitor startup durations.
- Review probe failures in logs.
- Adjust based on real-world behavior.
- Validate during rolling updates.
- Ensure no hidden failure states.

ARCHITECTURE DECISION

Use startup probes to protect legitimate initialization, not to excuse unreliable startup behaviour.

INIT CONTAINER FAILURE BLOCKS THE APPLICATION

POD STATUS
Init: Error
Containers: Waiting
(Init: CrashLoopBackOff)

Init containers must complete successfully before the main containers start.
If an init container fails, the Pod never becomes Ready.

1. COMMON CAUSES

DATABASE MIGRATION FAILURE
Schema changes fail, or migration scripts return an error.

DEPENDENCY UNAVAILABLE
Required service, API, or external system is down or unreachable.

PERMISSION ERRORS
Container does not have rights to read, write, or execute.

DNS RESOLUTION FAILURE
Cannot resolve hostnames for dependencies or internal services.

INCORRECT COMMAND
Syntax errors, wrong flags, or bad script logic cause immediate failure.

MISSING SECRET
Required secret or config not found or mounted.

ENDLESS RETRY LOOPS
Script retries forever without backoff or exit condition.

NON-IDEMPOTENT INITIALIZATION
Repeating the init causes conflicts or duplicate resources.

2. INIT CONTAINER FAILURE WORKFLOW

INIT STATUS → Check Pod status to see which init container is failing.

INIT LOGS → Inspect logs for the failing init container.

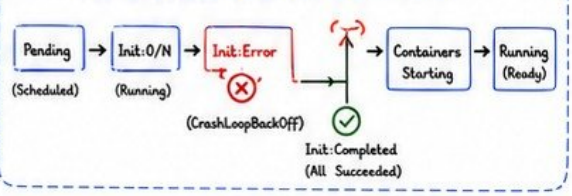
DEPENDENCY VALIDATION → Verify databases, APIs, DNS, secrets, permissions, etc.

RETRY BEHAVIOUR → Understand how the init container retries and when it gives up.

RECOVERY → Fix the root cause and allow the init to complete successfully.

KEY POINT
Until ALL init containers complete successfully, the application containers will never start.

POD LIFECYCLE WITH INIT CONTAINERS



3. INVESTIGATION COMMANDS

CHECK INIT STATUS

```
kubectl get pod <pod> -o wide
```

Look for INIT column (e.g., 0/3, 1/2, or Error)

VIEW INIT CONTAINER LOGS

```
kubectl logs <pod> -c <init-container-name>
# previous failed attempt
kubectl logs <pod> -c <init-container-name>
  --previous
```

DESCRIBE POD

```
kubectl describe pod <pod>
```

Check "Init Containers" section and events at the bottom.

CHECK POD CONDITIONS

```
kubectl get pod <pod>
-o jsonpath='{.status.conditions}'
```

QUICK CHECKLIST

- [x] Check init container status.
- [x] Read init container logs.
- [x] Validate dependencies (DB, API, DNS).
- [x] Verify secrets and config maps.
- [x] Check permissions and service accounts.
- [x] Confirm commands and scripts.
- [x] Test idempotency (run multiple times).
- [x] Fix the root cause and re-deploy.

4. BEST PRACTICES FOR INIT CONTAINERS

- Make init tasks idempotent and safe to re-run.
- Set reasonable timeouts and retry limits.
- Fail fast on unrecoverable errors.
- Log clearly with meaningful error messages.
- Validate all external dependencies before running migrations.
- Use least-privilege permissions and proper secrets.

5. EXAMPLE: INIT CONTAINER FAILURE

EVENTS (from kubectl describe pod)

Warning	BackOff	45s	kubelet	Back-off restarting failed container migrate
Warning	Failed	45s	kubelet	Error: could not connect to database

INIT CONTAINER LOG (migrate)

```
ERROR:  relation "users" already exists
DETAIL:  SQL state 42P07
HINT:    Make migration idempotent.
```

6. RECOVERY PATH

Identify failing init container → Inspect logs & root cause → Fix configuration, script, or dependency → Re-run / redeploy the Pod → Init completes → Pod starts

PRODUCTION IMPACT

- Application never starts
- Traffic goes to zero
- Rolling updates get stuck
- Deployments appear "hung"
- User-facing outage

SENIOR ENGINEER TIP

Init containers are powerful, but risky. Treat them like production code: Test, log, validate, and make them resilient and idempotent.
A single failing init can take down your entire deployment.

CONFIGMAP CHANGE BREAKS PRODUCTION

ConfigMaps are powerful — but a single wrong value can break your entire application.

1. COMMON WAYS CONFIGMAP CHANGES BREAK APPS

INVALID CONFIGURATION VALUES
Bad syntax, wrong data type, or unsupported values cause the application to fail.

INCORRECT ENVIRONMENT VARIABLES
Wrong variable name or value → app can't connect or start.

MISSING KEYS
Application expects a key that no longer exists in the ConfigMap.

CONFIG PARSING FAILURES
Application fails when reading files (YAML, JSON, properties, etc.).

MOUNTED FILE UPDATE BEHAVIOUR
Mounted ConfigMap files update on disk, but apps may not reload automatically.

PODS NOT AUTOMATICALLY RESTARTING
Pods keep running with the old config in memory.

CONFIGURATION DRIFT BETWEEN REPLICAS
Not all Pods get the same change at the same time.

UNSAFE CONFIGURATION ROLLOUT
No validation, no canary, no rollback → outage.

2. HOW CONFIGMAP CHANGES AFFECT APPLICATIONS

CONFIGMAP UPDATED → FILES UPDATED IN POD VOLUME → APPLICATION DOES NOT RELOAD → ERRORS IN LOGS → IMPACT ON USERS

Change on disk ≠ Change in memory
Your app must reload, or the Pod must restart.

3. INVESTIGATION WORKFLOW

1 CHECK EVENTS
`kubectl describe pod <pod>`
Look for mount, env, or startup errors.

2 REVIEW LOGS
`kubectl logs <pod>`
Look for config parsing or connection errors.

3 COMPARE CONFIG
`kubectl get configmap <name> -o yaml`
Compare with previous version / desired state.

4 VALIDATE APP
Test config outside the cluster if needed.
Use linters or schema validation.

5 ROLL OUT FIX
Update ConfigMap safely and roll out new Pods.

Always prove that the new configuration is valid before rolling it out to production.

4. USEFUL COMMANDS

GET CONFIGMAP
```
kubectl get configmap <name>
kubectl get configmap <name> -o yaml
```

DESCRIBE CONFIGMAP
```
kubectl describe configmap <name>
```

SEE WHAT POD USES IT
```
kubectl get pod -o jsonpath=
"{..volumes[*].configMap.name}"
kubectl get deploy -o yaml |
grep -B5 -A5 configmap
```

CHECK MOUNTED FILE CONTENT
```
kubectl exec -it <pod> --
cat /etc/config/app.conf
```

WATCH POD RESTARTS
```
kubectl get pods -w
kubectl rollout history
deployment/<name>
```

5. CONFIG UPDATE BEHAVIOUR

MOUNT TYPE	BEHAVIOUR	NEED RESTART?
Environment Variables	Loaded once at container startup	YES
Mounted Files	Updated on disk (eventually) when ConfigMap changes	MAYBE* App must reload or watch file
Command Arguments	Loaded at container startup	YES

*Not all apps watch for file changes automatically.

6. EXAMPLE: BAD CONFIG CHANGE

CONFIGMAP (OLD)
```
database:
  host: db.prod.svc.cluster.local
  port: "5432"
  poolSize: "20"
```

CONFIGMAP (UPDATED)
```
database:
  host: db.prod.svc.cluster.local
  port: "54320"     # Typo!
  poolSize: "2000"  # Too high!
```

APPLICATION LOG
```
ERROR   Failed to connect to database: connection refused
ERROR   HikariPool-1 - Connection is not available, request timed out
ERROR   Application startup failed
```

One wrong value → all Pods fail → users impacted.

7. BEST PRACTICE: TREAT CONFIG CHANGES AS RELEASES

VALIDATE FIRST
Use schema / linting and test in lower environment.

VERSION YOUR CONFIG
Track versions in Git. Use naming conventions.

OWNERSHIP
Every ConfigMap must have an owner.

SAFE ROLLOUT
Use canary / blue-green or rolling updates with monitoring.

ROLLBACK PLAN
Keep last known good version and rollback quickly.

SENIOR ENGINEER TIP
Configuration is code.
Manage it with the same rigor:
review, test, version, monitor, rollback.
No exceptions.

REMEMBER
- ☑ Small config change
- ☑ Big impact
- ☑ Validate
- ☑ Roll out safely
- ☑ Monitor
- ☑ Rollback fast

VERIQTA

SECRET FAILURE PREVENTS WORKLOAD STARTUP

POD STATUS
Init: CrashLoopBackOff
Reason: SecretError

Secrets power your applications – but any problem with secrets can stop your workload from starting.

1. SECRET FAILURE SCENARIOS

1. **MISSING SECRET**
The secret does not exist in the namespace.

2. **INCORRECT SECRET KEY**
The key referenced in the secret does not exist.

3. **EXPIRED CREDENTIALS**
Passwords, tokens, or certificates have expired.

4. **INVALID CERTIFICATE**
Certificate is expired, not yet valid, or issued for wrong domain.

5. **INCORRECT ENCODING**
Base64 or binary content is malformed or double-encoded.

6. **SECRET ROTATION MISMATCH**
App uses old secret version while service expects new credentials.

7. **EXTERNAL SECRET SYNC FAILURE**
Secrets manager / Operator failed to sync latest secret.

8. **WORKLOAD IDENTITY MISCONFIG**
ServiceAccount, IAM role, or workload identity not configured correctly.

2. HOW SECRET FAILURES AFFECTS PODS

POD STARTS → SECRET ACCESS FAILS → INIT / APP FAILS → RESTART LOOP → WORKLOAD UNAVAILABLE

- **POD STARTS**: Container initializes and needs secret at startup
- **SECRET ACCESS FAILS**: Pod cannot read or validate the required secret
- **INIT / APP FAILS**: Application exits or init container fails
- **RESTART LOOP**: Kubelet restarts pod (CrashLoopBackOff)
- **WORKLOAD UNAVAILABLE**: No traffic served

Many secret issues appear as: CrashLoopBackOff, ImagePullBackOff, Init:Error or SecretError.

3. INVESTIGATION WORKFLOW

1. **CHECK EVENTS**
kubectl describe pod <pod>
Look for secret related errors.

2. **VERIFY SECRET EXISTS**
kubectl get secret <secret-name> -n <namespace>

3. **CHECK SECRET CONTENT**
kubectl get secret <secret-name> -n <ns> -o yaml

4. **VALIDATE SERVICE ACCOUNT / ROLE**
kubectl describe sa
kubectl get rolebinding
Check permissions.

5. **CHECK EXTERNAL SECRET SYNC**
Check Operator / Secret Manager logs and status.

4. USEFUL COMMANDS

DESCRIBE POD
kubectl describe pod <pod>
Check Events and Init Containers for secret errors.

GET SECRET
kubectl get secret <secret-name> -n <ns>
Verify secret exists in the namespace.

VIEW SECRET (YAML)
kubectl get secret <secret-name> -n <ns> -o yaml
Inspect keys and metadata.

CHECK MOUNTED VOLUME
kubectl describe pod <pod>
Look for volumes and mounts using the secret.

CHECK EXTERNAL SECRETS
kubectl get externalsecret -A
kubectl get secrets -A
Check sync status.

SECURITY REMINDER

Never expose secret values while troubleshooting through:
- ✗ Screenshots
- ✗ Shell history
- ✗ Logs
- ✗ Incident channels

Protect secrets. Protect production.

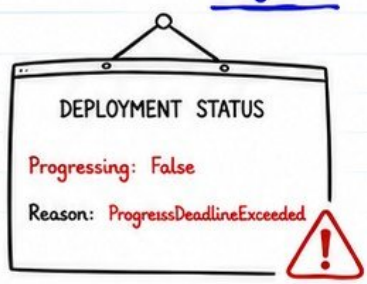
5. BEST PRACTICES

- ☑ Use least-privilege access for secrets.
- ☑ Rotate secrets regularly and validate after rotation.
- ☑ Use external secret management (AWS Secrets Manager, HashiCorp Vault, etc.).
- ☑ Enable monitoring and alerts for secret sync and access failures.
- ☑ Document secret dependencies and ownership.
- ☑ Test secret access in lower environments before production rollout.

VERIQTA

DEPLOYMENT ROLLOUT FAILURE

When a deployment update does not complete successfully



Kubernetes deployments are designed to update your applications safely. But misconfigurations or bad releases can cause the rollout to fail and block updates.

IMPACT

- New version never goes live
- Old version may be unavailable
- Services become slow or unreachable
- Future deployments get blocked

1. COMMON CAUSES

NEW PODS NEVER BECOME READY
Pods fail readiness or liveness checks, crash, or cannot start.

MAXUNAVAILABLE IS TOO AGGRESSIVE
Too many pods are taken down at once, causing service disruption.

MAXSURGE EXCEEDS CLUSTER CAPACITY
Not enough resources to schedule extra pods, so new pods remain pending.

BROKEN IMAGE OR CONFIGURATION
Invalid image, wrong tag, or bad config causes pods to fail.

PROBE FAILURES
Liveness or readiness probe misconfiguration causes continuous restarts.

PODDISRUPTIONBUDGET CONFLICTS
PDB prevents eviction of old pods, blocking the rollout.

DEPLOYMENT PROGRESS DEADLINE EXCEEDED
Kubernetes cannot complete the rollout within the specified progress deadline.

2. HOW A ROLLOUT WORKS (IDEAL FLOW)

START ROLLOUT → New ReplicaSet is created.

CREATE NEW PODS → New pods are scheduled and start.

READINESS CHECK → Pods must pass readiness probes.

TRAFFIC SWITCH → Service routes traffic to new pods.

ROLLOUT COMPLETE → Old pods are scaled down and removed.

3. WHEN ROLLOUT FAILS

NEW PODS FAIL → Pods crash, fail probes, or stay pending.

NOT ENOUGH READY PODS → Min available pods cannot be maintained.

PROGRESS STOPS → No new pods become ready within the deadline.

ROLLOUT MARKED FAILED → Deployment status = Progressing: False

Once failed, further deployments may be blocked until the issue is resolved or the deployment is rolled back.

4. SENIOR INVESTIGATION COMMANDS

COMMAND	PURPOSE
`kubectl rollout status deployment/<name> -n <ns>`	Watch the rollout in real-time. Shows progress and failures.
`kubectl rollout history deployment/<name> -n <ns>`	View revision history and see what changed.
`kubectl describe deployment <name> -n <ns>`	Check events, conditions, strategy, and failure reasons.
`kubectl get rs -n production`	List ReplicaSets and their desired vs current replicas.
`kubectl get pods -n production -l app=<app> -o wide`	Check pod statuses, node placement, and readiness.

5. QUICK CHECKLIST

- ☑ Check deployment events
- ☑ Verify new pod logs and describe output
- ☑ Confirm images, configs, and environment variables
- ☑ Validate resource requests/limits
- ☑ Review probe configuration
- ☑ Check PDBs and cluster capacity
- ☑ Compare current vs previous working revision

RECOVERY OPTIONS

- Roll back to last known good revision
- Fix the root cause
- Re-run the deployment
- Increase resources or adjust strategy

BEST PRACTICES

- Use canary or blue/green deployment
- Set a reasonable progressDeadlineSeconds
- Test in staging before production
- Monitor rollout with alerts
- Keep rollback simple and fast

SENIOR ENGINEER TIP

A failed rollout is a symptom, not the problem. Focus on why the new pods cannot become ready. Fix the cause, not just the deployment.

PRODUCTION PRINCIPLE

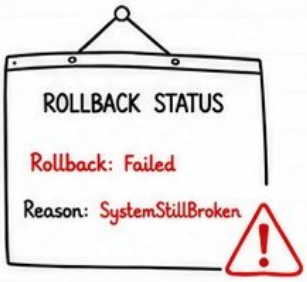
Safe deployments protect users, protect the platform, and build trust in every release.

SECURITY REMINDER

Never expose sensitive information while troubleshooting (env vars, secrets, tokens, certificates) in screenshots, logs, or chats. Protect data. Protect production.

VERIQTA

FAILED ROLLBACK AND MIXED APPLICATION VERSIONS



Rolling back a deployment does not always return your system to the previous stable state.

1. WHY ROLLBACKS OFTEN FAIL

ROLLBACK DOES NOT RESTORE EXTERNAL DEPENDENCIES
Databases, queues, caches, and third-party services may already be changed.

DATABASE SCHEMA INCOMPATIBILITY
New application version may have modified the schema. Old version cannot work with new schema.

CONFIGMAP AND SECRET CHANGES REMAIN
Rollback does not revert updated ConfigMaps or Secrets automatically.

OLD REPLICASET NO LONGER HAS A VALID IMAGE
Image might be deleted, garbage collected or no longer accessible.

MUTABLE IMAGE TAGS
Using :latest or mutable tags means the old version is not the same anymore.

PARTIAL TRAFFIC ACROSS INCOMPATIBLE VERSIONS
LoadBalancer or Service may still route traffic to new pods.

SESSION AND CACHE INCOMPATIBILITY
Sessions, tokens or cache entries created by new version may break old version.

2. WHAT HAPPENS DURING A FAILED ROLLBACK

NEW RELEASE (v2)
v2 Pods
- New code
- New schema
- New config

FAILS →

ROLLBACK INITIATED
v1 Pods (Old)
- Old code
- Incompatible with new schema and data

→

MIXED ENVIRONMENT
- v1 Pods + v2 Pods
- New ConfigMaps still active
- New Secrets still active
- Schema already changed

→

SYSTEM STILL BROKEN
Rollback did not restore the system to a working state.

Rollback reverts the deployment, not the entire system. Stateful changes remain.

3. COMMON SIGNS OF A BROKEN ROLLBACK

- Old pods crash or fail readiness
- Database errors due to schema mismatch
- Config/Secret values still incompatible
- Intermittent errors due to mixed traffic
- Sessions invalid after rollback
- Cache deserialization errors
- Some users see errors, some don't

4. INVESTIGATION COMMANDS

CHECK ROLLOUT STATUS
```
kubectl rollout status deployment/<name> -n <ns>
```

VIEW ROLLOUT HISTORY
```
kubectl rollout history deployment/<name> -n <ns>
```

DESCRIBE DEPLOYMENT
```
kubectl describe deployment <name> -n <ns>
```

CHECK REPLICASETS
```
kubectl get rs -n <ns>
```

CHECK PODS AND VERSIONS
```
kubectl get pods -n <ns> -o wide --show-labels
```

5. HOW TO PREVENT FAILED ROLLBACKS

- ☑ Use immutable image tags (e.g., v1.2.3)
- ☑ Version your ConfigMaps and Secrets
- ☑ Use database backward-compatible migrations
- ☑ Test rollback in staging before production
- ☑ Automate dependency versioning
- ☑ Keep old images for rollback (set retention policies)
- ☑ Use feature flags for risky changes
- ☑ Document and rehearse rollback procedures

DESIRED STATE AFTER ROLLBACK

Application (v1) + Database (Compatible) + ConfigMaps (v1) + Secrets (v1) = System Stable and Consistent

REAL WORLD LESSON

Application rollback is not necessarily system rollback. Databases, queues, caches, configuration, and external services may remain changed. Always roll back the environment, not just the deployment.

VERIQTA

WORKLOAD OUTAGE RESPONSE PLAYBOOK

OUTAGE STATUS
Status: ACTIVE
Severity: HIGH

A clear and calm response saves time, reduces impact, and prevents small issues from becoming major outages.

1. INCIDENT FLOW (FOLLOW THE ORDER)

01 ALERT — Receive alert and acknowledge immediately.

02 VERIFY IMPACT — Who is affected? What is broken? User impact?

03 FREEZE CHANGES — Stop deployments and configuration changes.

04 INSPECT ROLLOUT — Check deployment status and rollout history.

05 INSPECT PODS — Check pod status, restarts, and readiness.

06 CHECK EVENTS — Look for warnings, errors and scheduling issues.

07 REVIEW LOGS — Inspect container logs and application logs.

08 VALIDATE RESOURCES — Check CPU, memory, disk, and limits.

09 VALIDATE DEPENDENCIES — Databases, queues, APIs, caches, and external services.

10 RECOVER — Apply the right recovery action (see section 2).

11 MONITOR — Watch metrics, logs, and alerts closely.

Stay calm. Follow the process. Communicate clearly.

2. RECOVERY CHOICES (CHOOSE THE RIGHT ACTION)

ROLL BACK	Roll back to the last known good version. Use: kubectl rollout undo deployment/<name>
SCALE HEALTHY REPLICAS	Scale up healthy replicas to restore capacity. Use when some pods are still working.
RESTORE CONFIGURATION	Revert ConfigMaps or environment variables to the last known good state.
CORRECT SECRET REFERENCES	Fix secret names, keys or mounts. Redeploy to apply changes.
DISABLE A HARMFUL PROBE SAFELY	Temporarily relax or disable a failing probe that is killing healthy pods.
REDIRECT TRAFFIC	Route traffic away from the failing service or version (Ingress/Service/LoadBalancer).
APPLY AN EMERGENCY PATCH	Hotfix the issue and redeploy. Document the change immediately.
ESCALATE TO OWNERS	Escalate to application or platform owners if the issue is outside your scope.

3. SENIOR INVESTIGATION COMMANDS

```
kubectl get pods -n <ns> -o wide
```
List pods and see their nodes and IPs.

```
kubectl describe pod <pod> -n <ns>
```
Deep dive into pod status, events and reasons.

```
kubectl rollout status deployment/<name> -n <ns>
```
Check rollout progress and detect failures.

```
kubectl rollout history deployment/<name> -n <ns>
```
Review previous revisions and changes.

```
kubectl get events -n <ns> --sort-by=.lastTimestamp
```
See recent events in chronological order.

```
kubectl top pods -n <ns>
```
Check real-time CPU and memory usage.

4. COMMUNICATION CHECKLIST

- [x] Update incident channel with facts, not assumptions.
- [x] Share impact, steps taken and next actions.
- [x] Communicate recovery steps before applying.
- [x] Close the loop when the service is stable.

PRO TIP

Document the incident after recovery:

- [x] What happened
- [x] Root cause
- [x] How it was fixed
- [x] How to prevent it next time

VERIQTA PRODUCTION PRINCIPLE

Follow the process. Verify before you act. Measure impact. Then recover with confidence.

Document everything. Learn every time. Prevent the next outage.

KEY TAKEAWAY

Speed comes from having a process, not from guessing.
Follow the playbook.
Protect the users.

VERIQTA
VOLUME 1
PRODUCTION CHECKLIST

CHECKLIST STATUS
- ☑ Use before every deployment
- ☑ Review during every outage

This checklist helps you deploy safely, respond quickly, and operate reliably in production.

1. BEFORE DEPLOYMENT

- ☑ **VALIDATE MANIFESTS**
  Run kubectl apply --dry-run=server and kubectl diff. Fix issues early.
- ☑ **PIN IMMUTABLE IMAGE VERSIONS**
  Use specific tags or digests. Never use :latest in production.
- ☑ **TEST PROBES**
  Liveness, readiness, and startup probes must be accurate and reliable.
- ☑ **CONFIRM RESOURCE SIZING**
  CPU/Memory requests & limits based on real usage and headroom.
- ☑ **VALIDATE CONFIGMAPS AND SECRETS**
  Check keys, values, mounts, and references. Validate in a staging environment.
- ☑ **TEST ROLLBACK PROCEDURES**
  Ensure rollback works. Know your previous stable version.
- ☑ **CHECK DISRUPTION BUDGETS**
  PDBs prevent too many pods from going down at once.
- ☑ **CONFIRM OBSERVABILITY**
  Logs, metrics, and alerts must be visible and working end-to-end.

2. DURING AN OUTAGE

- ☑ **PROTECT EVIDENCE**
  Do not delete resources immediately. Evidence helps find the root cause.
- ☑ **INSPECT EVENTS BEFORE DELETING PODS**
  kubectl get events --sort-by=.lastTimestamp shows what happened and when.
- ☑ **CAPTURE PREVIOUS CONTAINER LOGS**
  kubectl logs <pod> -c <container> --previous captures logs from crashed containers.
- ☑ **REVIEW RECENT CHANGES**
  Check deployments, config changes, secrets, and infrastructure updates.
- ☑ **SEPARATE SYMPTOMS FROM ROOT CAUSES**
  Don't fix the symptom. Find the real cause of the failure.
- ☑ **RECOVER SERVICE BEFORE DEEP ANALYSIS**
  Restore user impact first, then perform detailed investigation.
- ☑ **DOCUMENT EVERY EMERGENCY ACTION**
  Record what you did, why, and the outcome for learning and audit.

3. QUICK REFERENCE COMMANDS

Get events
```
kubectl get events --sort-by=.lastTimestamp
```

Describe pod
```
kubectl describe pod <pod> -n <ns>
```

Logs (previous)
```
kubectl logs <pod> -c <container> --previous
```

Top pods
```
kubectl top pods -n <ns>
```

Rollout status
```
kubectl rollout status deployment/<name> -n <ns>
```

Get all pods
```
kubectl get pods -n <ns> -o wide
```

FINAL PRINCIPLE

A pod status is only one signal.
Senior engineers correlate Kubernetes state, application behaviour, infrastructure health, dependency status, and user impact before deciding what failed.

REMEMBER

- ★ Deploy small, verify, then scale.
- ★ Observe everything.
- ★ Automate what you can.
- ★ Document what you change.
- ★ Learn after every incident.